

Musterlösung — Übungsklausur 9

Deep Learning 1 · TU Berlin

1 Multiple Choice

1. Hyperbolic Secant → welcher Loss?

✓ (c) Log-Cosh Loss

- (a) Squared
- (b) Absolute
- (d) Cross-Entropy

2. Vorteil Sparse Coding?

✓ (b) Geringe Speicherkosten (Index-Wert-Paare) und hohe Interpretierbarkeit

- (a) Rechenkosten
- (c) Bessere Approximation
- (d) Keine Regularisierung

3. Translation Equivariance?

✓ (a) $f(T(x)) = T(f(x))$

- (b) Invarianz
- (c) Translation als Feature
- (d) Nur 1D

4. TCAV in XAI?

✓ (b) Identifiziert Mid-Level-Konzepte (z.B. Streifen) die Vorhersagen erklären

- (a) Activation Maximization
- (c) LRP-Variante
- (d) Adversarielle Angriffe

5. Shortcut Connections im Autoencoder?

✓ (a) Low-Level-Information fließt direkt zum Decoder ohne Bottleneck

- (b) Ersetzen Encoder
- (c) Erhöhen Kompression
- (d) Reduzieren Trainingszeit

6. Problem bei Ensemble-Uncertainty?

✓ (b) Hoher Rechenaufwand durch Training mehrerer Modelle

- (a) Kann keine Unsicherheit schätzen
- (c) Keine Theoriebasis
- (d) Nur Klassifikation

7. Eigenwerte symmetrischer Matrix?

✓ (a) $\lambda_{\max} = \max_{\|v\|=1} v^T A v$, $\lambda_{\min} = \min_{\|v\|=1} v^T A v$

- (b) Immer gleich groß
- (c) $\lambda_{\max}$ positiv, $\lambda_{\min}$ negativ
- (d) Produkt der Diagonalelemente

8. Aktivierungsfunktion LSTM Gates?

✓ (c) Sigmoid — produziert Werte in (0,1) für Gating

- (a) ReLU
- (b) Tanh (für cell state, nicht gates)
- (d) Softmax

2 Theorie — Lineares RNN & Pathologische Gradienten

(a) Lineares RNN als Matrixgleichung

$$\begin{pmatrix} y_i & h_i \end{pmatrix} = \begin{pmatrix} A & B \\ C & D \end{pmatrix} \cdot \begin{pmatrix} x_i & h_{i-1} \end{pmatrix}$$

A: $x \rightarrow y$ Gewichte. B: $h \rightarrow y$ Gewichte. C: $x \rightarrow h$ Gewichte. D: $h \rightarrow h$ Gewichte (wichtig für Gradientenverhalten).

(b) Exploding/Vanishing

Gradient zur Zeit t enthält Produkt D^k (k Zeitschritte zurück). Größter Eigenwert $\lambda_{\max}$ von D :

$$\lambda_{\max} > 1: \|D^k\| \rightarrow \infty \rightarrow \text{Exploding Gradients}$$

$$\lambda_{\max} < 1: \|D^k\| \rightarrow 0 \rightarrow \text{Vanishing Gradients}$$

Beide verhindern Lernen von Long-Range-Dependencies.

(c) Gegenmaßnahmen

1. Gradient Clipping: $g \leftarrow g \cdot \min(1, \text{threshold}/\|g\|)$. Verhindert Exploding Gradients.
2. Rauschen auf Hidden States (Noise Injection): Zwingt Modell zur Desensibilisierung gegenüber kleinen Schwankungen $\rightarrow$ reduziert Exploding-Gradient-Risiko.

3 Theorie — Domain Adaptation

(a) Problem bei verschiedenen Domänen

Starker Distributionsdrift zwischen Subgruppen verschiedener Domänen: Modell, das auf Domäne P trainiert wurde, versagt auf Domäne Q . Test-Accuracy auf einer Domäne gibt kaum Info über Performance auf der anderen.

(b) Lösung 1: Regularisierung der Domänenausgaben

$$L_{\text{total}} = L_{\text{task}} + \lambda \cdot (E_P[\text{model}(x)] - E_Q[\text{model}(x)])^2$$

Zusätzlicher Term erzwingt, dass das Modell auf beiden Domänen im Durchschnitt dieselben Antworten produziert $\rightarrow$ reduziert Domänensensitivität.

(c) Lösung 2: Adversarielle Domänenkritik

Zwei Netze: (1) Feature Extractor ϕ + Task Predictor. (2) Domain Critic (Klassifikator der Domäne).

Training: Feature Extractor wird so trainiert, dass Domain Critic die Domäne nicht mehr unterscheiden kann (Gradient Reversal Layer). Task Predictor optimiert Task-Loss. Resultat: Features die keine Domäneninformation enthalten $\rightarrow$ Domäneninvarianz.

4 Theorie — Mixture Density Networks

(a) Inverses Problem

Problem: Zwei oder mehr verschiedene Inputs können zum gleichen Output führen $\rightarrow$ Abbildung nicht invertierbar. Squared Loss würde den Durchschnitt aller Lösungen vorhersagen $\rightarrow$ physikalisch falsch / Artefakt.

Lösung: Bedingte Wahrscheinlichkeitsverteilung $p_{\theta}(t|x)$ modellieren, nicht Punktschätzung.

(b) MDN Formel

$$p(t|x) = \sum_{i=1}^m \alpha_i(x) \cdot N(t; \mu_i(x), \sigma_i^2(x))$$

m: Anzahl Gaußkomponenten. α_i : Mischungsgewichte. μ_i : Mittelwerte. σ_i : Standardabweichungen. Alle durch ein NN aus x berechnet.

(c) Planeten-Beispiel

Forward Map: Planetenzusammensetzung (Fe, Si, H₂O-Anteile) $\rightarrow$ Masse, Radius (deterministisch aus physikalischen Gesetzen).

Inverse Map: Masse + Radius $\rightarrow$ Zusammensetzung. Viele Zusammensetzungen können zur gleichen (Masse, Radius) führen $\rightarrow$ MDN modelliert multimodale Verteilung über mögliche Zusammensetzungen.

Trainiertes MDN: gibt für unbekannte Planeten eine Wahrscheinlichkeitsverteilung über mögliche innere Strukturen.

5 Programmierung — RNN Training

(a) BPTT

BPTT: Behandle RNN als tiefes Netz (ein Layer pro Zeitschritt). Gradient des Loss $E = \sum_t \ell(y_t, \text{target}_t)$ wird via Standard-Backpropagation durch alle Zeitschritte berechnet. Kettenregel erzeugt Produkte von Jacobi-Matrizen $\rightarrow$ Vanishing/Exploding Gradients.

$$E = \sum_{t=1}^T \ell(y_t, \text{target}_t)$$

(b) Manuelles RNN in PyTorch

```
w_h = nn.Parameter(torch.randn(d_h, d_h))
w_x = nn.Parameter(torch.randn(d_h, d_x))
h0 = torch.zeros(batch, d_h)
h1 = torch.tanh(x1 @ w_x.T + h0 @ w_h.T)
h2 = torch.tanh(x2 @ w_x.T + h1 @ w_h.T)
h3 = torch.tanh(x3 @ w_x.T + h2 @ w_h.T)
y = h3 # oder weitere Output-Schicht
loss = nn.MSELoss()(y, target)
loss.backward() # BPTT automatisch via autograd
```